

Skorpio

Agentic AI for the people who plan Canada's grid

"Ask a hard question. Watch the AI think. Get a grounded answer."

Technical Overview

Hackathon Submission. Seneca Hackathon 2026.

Track: Smart Grid Resilience and Preparing for Electrification

Team members:

Christian Rafael. Fawad Arshad. Krish Barot. Maximilian Ali. Vansh Barot.

Demo: skorpio-xjenosa-s-projects.vercel.app

Reports: tinyurl.com/skorpio-reports

Source: github.com/xjenosa/Skorpio

Table of Contents

Executive Summary

The Problem We Set Out to Solve

Our Solution: Five Agentic Pipelines

End-to-End System Architecture

Technology Stack

Backend Deep Dive (FastAPI + Orchestrator)

The Five Pipelines, in Detail

AI Inference: Claude Tool Use, Streaming, and Counterfactuals

Data Sources and Grid Intelligence

Frontend: React, Vite, Mapbox, and Live Reasoning UI

Database, Auth, and Storage

Observability and Reliability

Engineering Trade-offs in 72 Hours

Roadmap and Post-Hackathon Plans

Team, Credits, and Closing Notes

About this document

This overview describes Skorpio as built during the Seneca Hackathon 2026 sprint. It covers the architecture, the engineering decisions, the trade-offs we accepted in 72 hours, and where the project goes next. Code references throughout point to actual files in the repository.

Executive Summary

Skorpio is an agentic AI platform that helps the people who plan Canada's electricity grid answer hard questions in minutes instead of weeks. A planner types a natural-language question. Claude reads it, routes it to one of five purpose-built pipelines, loads real Canadian grid data, and produces a printed-quality report with citations to source feeds. The planner can watch every stage of the reasoning happen live, then ask follow-up questions in plain English against the same report.

Everything is built around three commitments. Real data, not synthetic. Visible reasoning, not a black box. And five tools that share one brain, so a planner never has to learn five different UIs.

The platform ships as a containerised stack (FastAPI API, async orchestrator, Postgres) and a React single-page app deployed on Vercel. The agent layer runs on Anthropic's Claude family with the operator's model of choice (Opus 4.7 for synthesis, Sonnet 4.6 for routine workloads, Haiku 4.5 for fast classification). Live grid data comes from IESO, AESO, ECCC, OEB, StatsCan, OSM, ArcGIS, and NRCan, normalised into a single internal feature layer. Five pipelines cover the five questions Canadian utilities actually have to answer right now: where to put a new datacenter, how to grow an existing one, whether the local grid survives the next cold snap, which neighbourhoods are ready for electrification, and where to spend a fixed hardening budget for the most climate-resilience per dollar.

The Problem We Set Out to Solve

Canada's grid was sized for a world that no longer exists. Three pressures are arriving at once. Climate. Electrification. Scale.

Thirty percent of Canadian homes are projected to switch to electric heat pumps by 2030. Polar vortex events arrive harder and last longer than they did a decade ago. A new wave of AI training and inference workloads is asking for datacenter capacity at a pace and a power density the existing grid was never designed to absorb. When one of these stresses hits, a planner at a Canadian utility has about twenty-four hours to figure out where the grid will break. Today, they do that work in spreadsheets. By hand.

What is broken about the existing options

- Existing planning tools are siloed: one for datacenter siting, another for winter peak modelling, a third for distribution feeder analysis. None of them share data, and most of them require a domain expert to operate.
- Off-the-shelf load forecasts use national averages. They miss neighbourhood-scale realities: which streets are 1970s baseboard heat, where the nearest substation is over-allocated, which feeders are already at 80% summer load.
- Generic LLMs hallucinate numbers. A planner cannot hand a stakeholder a report whose Megawatts came from a model's training data.
- Investment optimisation is done with a stack of PDFs and a whiteboard. The connection between climate hazard exposure, asset condition, and ROI is rebuilt from scratch every budget cycle.

Design principle

If a planner cannot point at every number in the report and trace it to a real public data source, we have failed. Provenance, visible reasoning, and grounded answers matter more than vocabulary breadth at this stage.

Our Solution: Five Agentic Pipelines

Skorpio's surface area is intentionally small. A single chat-style composer where a planner types a question. Behind that one input, a Router agent classifies the question into one of five pipelines, and the right pipeline runs end-to-end without further configuration.

The five pipelines were chosen to cover the question space Canadian utilities and grid operators are actually living with in 2026. Each is a multi-stage agentic flow, each runs on live data, and each ends in a printable report with follow-up chat scoped to its own numbers.

Pipeline	Question it answers
Datacenter Siting	Where in Canada should we build a new AI or HPC datacenter, ranked by carbon intensity, latency, and total cost of energy?
Datacenter Expansion	How do we grow an existing operator's footprint with brownfield phases at sites they already own plus one or two greenfield neighbours?
Winter Peak Stress Tester	Will our local grid hold up at minus twenty-five degrees, once thirty percent of homes have heat pumps?
Electrification Readiness	Which neighbourhoods in our service territory are ready for electric vehicles and heat pumps, and which need help first?
Grid Investment Optimiser	If we have a fixed upgrade budget, how do

	we spend it for the most climate-resilience per dollar?
--	---

How a single run feels

A planner opens Skorpio and types into the composer. The Router agent reads the question, picks Winter Peak Stress Tester, and the dashboard switches to the live stages view. Five labelled cards light up one at a time in orange as the agent works. Stage one parses the request into a structured spec. Stage two loads the real grid topology for the region from OSM and OEB territory polygons. Stage three runs an hour-by-hour cold-event simulation. Stage four ranks every feeder by failure risk. Stage five composes the report.

When the report lands, it is not a wall of text. It is a verdict card on the left (peak number, headroom, at-risk feeder count, PASS or FAIL), a map of the local feeders coloured by risk, a stack of charts, and a recommendations table. Underneath, a chat bar lets the planner ask follow-up questions like "what if heat pump adoption hits 50%?" or "which upgrade gives us the most resilience for the money?". Every answer is grounded in the simulation that just ran.

End-to-End System Architecture

Skorpio splits cleanly into three planes. A React single-page app on Vercel, a FastAPI backend running uvicorn, and Anthropic's Claude API. The backend hosts both the synchronous HTTP surface and the asynchronous orchestrator that drives multi-stage pipeline runs. The frontend talks to the backend through a typed HTTP client and subscribes to a Server-Sent Events stream for live stage progress.

Request flow (worked example: Winter Peak)

```
Browser (React)
-> POST /api/winter-peak { query, model? }
   routers/pipelines.py validates the request
   runtime.spawn_pipeline_task() registers asyncio.Task in running_tasks
   returns 202 + job_id immediately

Task (asyncio)
-> SkorpioOrchestrator.run_winter_peak(job_id, spec)
   stage 1: WinterWorkloadAgent      -> structured WinterPeakSpec
   stage 2: WinterGridAgent          -> loaded topology + baseload
   stage 3: ColdEventSimAgent        -> hourly load curves
   stage 4: FeederRiskAgent          -> per-feeder risk ranking
   stage 5: WinterSynthesisAgent     -> ResiliencePlan JSON
   each stage: progress update written to Postgres jobs row

Browser (React)
<- GET /api/stream/{job_id} (Server-Sent Events)
   emits {stage, progress, message, paused, error} every 0.5s
   on complete: GET /api/jobs/{job_id} -> final report JSON
```

Why split sync vs async this way

- Pipeline runs take five to twenty minutes. Holding an HTTP connection open that long is fragile across Vercel edge timeouts and corporate proxies. Returning 202 with a job_id and streaming progress separately solves both.
- The orchestrator is in-process (not a separate worker pool). Pipelines are I/O-bound (Anthropic calls, HTTP fetches against IESO and ECCC), so asyncio gives us the concurrency we need without operational complexity.
- Pause and cancel are first-class. JobControl flags are checked at stage boundaries; mid-stage cancellation raises JobCancelledError and rolls progress back cleanly.
- The frontend's demo mode bundles five JSON fixtures and intercepts the same API client paths, so the Vercel deployment works even when the backend is unavailable.

Technology Stack

Frontend

- Runtime — React 18.3 + TypeScript 5.6, built with Vite 5.4.

- Maps — Mapbox GL 3.23 for the interactive candidate / feeder / hazard layers. Token via VITE_MAPBOX_TOKEN; falls back to a coordinate list if absent.
- Charts — Recharts 3.8 for load curves, readiness scores, ROI bars.
- Geospatial — topojson-client + world-atlas for province / territory overlays.
- Routing — React Router 6 with five pipeline-specific report routes.
- Demo plumbing — VITE_DEMO_ONLY env var locks the Vercel build into fixture mode.
- Deployment — Vercel (skorpio-xjenosa-s-projects.vercel.app). Built with vite build, served as a static site.

Backend

- API — FastAPI 0.115 + Uvicorn (Python 3.11).
- Orchestrator — Pure asyncio inside the API process; tasks tracked in a running_tasks dict, controllable via JobControl.
- Database — PostgreSQL 16 with SQLAlchemy 2 async + asyncpg driver. Alembic for migrations.
- Agents — Anthropic AsyncAnthropic SDK with three allowlisted models: claude-opus-4-7, claude-sonnet-4-6, claude-haiku-4-5-20251001.
- Geo — Shapely + GeoPandas for OEB territory polygons and OSM substation buffering.
- HTTP — httpx with timeouts and retry policy for live grid feeds.
- Observability — Structured request logging middleware; per-stage progress entries appended to the job's progress_log.
- Container — Docker Compose orchestrates three services: postgres, api, frontend (dev only — prod frontend is on Vercel).

AI / model layer

- Anthropic Claude (Opus 4.7, Sonnet 4.6, Haiku 4.5). Model picked per-stage: Haiku for the Router and routine classification; Sonnet for most workload parsing; Opus reserved for synthesis stages on long-context reports.
- Tool use — Five counterfactual tools (one per pipeline) exposed to the chat agent so users can re-run scenarios without leaving the report.
- Reconciliation — Post-stream pass that catches numeric drift in Claude's narration against the deterministic tool output.

Why these choices

- React + Vite over Next.js — Skorpio is a single-page app with one API origin; we did not need server-side rendering or App Router complexity. Vite gave us instant HMR throughout the sprint.
- FastAPI + pure asyncio over Celery — All pipeline work is I/O-bound to Anthropic and HTTP feeds. A separate broker plus worker process would have doubled the moving parts for no throughput gain inside a single tenant.

- Mapbox over Leaflet — We needed vector tiles and styling parity with the dark Claude-inspired UI. Mapbox token cost is low at hackathon traffic.
- Anthropic Claude — Mid-pipeline JSON extraction needs a model that obeys schema and rarely hallucinates units. Claude 4 family is the only one in our testing that held both bars at the same time.
- Postgres over SQLite — We needed ACID writes from the orchestrator while the API was serving reads, plus async drivers. Postgres in Docker is one container; the operational cost is negligible.

Backend Deep Dive (FastAPI + Orchestrator)

The backend is laid out as a conventional FastAPI app with explicit modules per concern. The module map follows the principle that each file owns one external dependency, one persistence boundary, or one named agent.

```
backend/  
  main.py                # FastAPI app, lifespan, CORS, router mounting  
  runtime.py             # Model allowlist, request schemas, task registry  
  orchestrator.py        # SkorpioOrchestrator – 5 run_* methods  
  orchestrator_stream.py # SSE progress streamer (0.5s poll, 15 min cap)  
  orchestrator_control.py # JobControl: pause / resume / cancel flags  
  auth.py                # Google OAuth + magic-code email JWT sessions  
  chat_tools.py          # Counterfactual tools + tool registry + reconciliation  
  config.py              # Pydantic Settings (.env loader)  
  agents/  
    base_agent.py        # AsyncAnthropic wrapper, retry, JSON parsing  
    router.py            # Pipeline classifier (Haiku + regex fallback)  
    grid_intelligence.py # Siting stage 1: region + grid scan  
    site_generation.py   # Siting stage 2: candidate generation  
    placement_scoring.py # Siting stage 3: multi-objective scoring  
    plan_synthesis.py    # Siting stage 4: SitingPlan synthesis  
    winter_workload.py   # Winter stage 1: WinterPeakSpec parsing  
    winter_grid.py       # Winter stage 2: feeder topology load  
    cold_event_sim.py    # Winter stage 3: hourly load curve sim  
    feeder_risk.py       # Winter stage 4: per-feeder risk ranking  
    winter_synthesis.py  # Winter stage 5: ResiliencePlan synthesis  
    electrification_workload.py / neighborhood_profile.py /  
    adoption_modeling.py / readiness_scoring.py /  
    electrification_synthesis.py # 5-stage electrification pipeline  
    investment_workload.py / asset_catalog.py /  
    climate_risk_modeling.py / portfolio_optimization.py /  
    investment_synthesis.py # 5-stage investment pipeline  
    expansion_workload.py / operator_footprint_agent.py /  
    demand_forecast.py / expansion_scoring.py /  
    expansion_synthesis.py # 5-stage expansion pipeline  
    grounding.py         # Citation enforcement for synthesis stages  
  routers/  
    pipelines.py         # 5 pipeline endpoints + cancel + resume  
    jobs.py              # Job CRUD, CSV import / export  
    live.py              # SSE progress endpoint  
    chat.py              # SSE-streamed follow-up chat per report  
    health.py            # /api/health  
    snowflake.py         # Optional analytics endpoint (deprecated)  
  grid/                  # Pure-Python simulators and scorers  
    winter_simulator.py / feeder_topology.py / electrification_modeling.py /  
    expansion_modeling.py / scoring_engine.py / generator.py /  
    filters.py / multi_objective.py  
  services/              # External data feeds + cached snapshots  
    eccc.py / iso_lmp.py / carbon_intensity.py / cold_events.py /  
    heat_pump_cop.py / ev_data.py / arcgis_enrichment.py /  
    arcgis_hazards.py / oeb_territory.py / nrcan_wildfire.py / ...
```

```

_ieso_hoep_generated.py / _aeso_pool_ail_generated.py /
_fsa_data_generated.py / _oeb_territories_generated.py / ...
db/
  models.py          # User, Job, EmailCode SQLAlchemy ORM
  crud.py            # Job + progress CRUD
langflow_components/ # Custom Langflow wrappers for siting pipeline
utils/              # Logging, time, formatting helpers
scripts/            # export_demo_fixtures.py and operational scripts

```

Lifespan and service wiring

```

@asynccontextmanager
async def lifespan(app: FastAPI):
    await Database.connect()
    init_snowflake_tables(optional=True) # skipped if creds missing
    yield
    await Database.disconnect()
    await cancel_running_pipelines()

```

The orchestrator is a module-level singleton. There is no separate worker process. Pipeline tasks are created with `asyncio.create_task` and parked in a process-wide `running_tasks` dict, keyed by `job_id`, so the cancel endpoint can find and stop them.

Endpoints

Method	Path	Purpose
GET	/api/health	Liveness + API key availability
POST	/api/site	Start a Datacenter Siting run
POST	/api/expansion	Start a Datacenter Expansion run
POST	/api/winter-peak	Start a Winter Peak Stress run
POST	/api/electrification	Start an Electrification Readiness run
POST	/api/investment	Start a Grid Investment Optimiser run
GET	/api/jobs/{job_id}	Fetch a job (spec, progress, result)
GET	/api/stream/{job_id}	SSE stream of stage progress
POST	/api/jobs/{job_id}/cancel	Cancel a running job
POST	/api/jobs/{job_id}/pause	Pause / resume a running job
POST	/api/chat/{job_id}	Follow-up chat scoped to a completed report
POST	/api/auth/google/login	Google OAuth entry point
POST	/api/auth/email/request-code	Send a 6-digit magic code to the user's email
POST	/api/auth/email/verify-code	Verify a code, return a JWT session

GET	/api/jobs.csv	Export jobs as CSV
POST	/api/jobs/import	Import jobs from a CSV with conflict resolution

Orchestrator stage pattern

Every pipeline follows a consistent shape so the frontend can render the same stage-by-stage UI for all five. The orchestrator method is responsible for stage sequencing, progress callbacks, and persisting both intermediate state and the final report.

```

async def run_winter_peak(self, job_id: str, spec: WinterPeakRequest):
    job_control = JobControl(job_id)
    try:
        await self._update(job_id, stage='workload', progress=5,
                           message='Parsing request')
        parsed = await WinterWorkloadAgent(self.client).run(spec.query)
        job_control.checkpoint()

        await self._update(job_id, stage='grid', progress=25,
                           message=f'Loading {parsed.region} feeder topology')
        grid = await WinterGridAgent(self.client).run(parsed)
        job_control.checkpoint()

        await self._update(job_id, stage='simulate', progress=50,
                           message='Simulating cold event hour by hour')
        curves = await ColdEventSimAgent().run(parsed, grid)
        job_control.checkpoint()

        await self._update(job_id, stage='risk', progress=75,
                           message='Ranking feeders by failure risk')
        risk = await FeederRiskAgent(self.client).run(grid, curves)
        job_control.checkpoint()

        await self._update(job_id, stage='synthesize', progress=90,
                           message='Composing report')
        plan = await WinterSynthesisAgent(self.client).run(parsed, grid, curves, risk)

        await self._finish(job_id, result=plan.model_dump())
    except JobCancelledError:
        await self._update(job_id, stage='cancelled', progress=0,
                           message='Cancelled by user')
    except Exception as exc:
        await self._fail(job_id, error=str(exc))

```

The Five Pipelines, in Detail

1. Datacenter Siting

Question: where in Canada should a new AI or HPC datacenter live, ranked by carbon intensity, latency to peering points, and total cost of energy?

Stages: GridIntelligenceAgent scans candidate regions and pulls live grid data.

SiteGenerationAgent generates parcels filtered by transmission proximity, hazard exposure, and permit feasibility. PlacementScoringAgent runs the multi-objective scorer (LCOE, gCO2/kWh, latency, capacity). PlanSynthesisAgent composes a SitingPlan with the top-N candidates, the trade-off table, and a narrative.

Data inputs: IESO HOEP and Generators, AESO Pool Price + AIL, provincial generation mix, ECCC weather, OEB territory polygons, OSM substations and transmission, PeeringDB latency, and the ArcGIS hazards layer.

Stages 1 to 4 of this pipeline are wrapped as custom Langflow components in `backend/langflow_components/` so the same logic can be reused in a visual builder.

2. Datacenter Expansion

Question: how does an existing datacenter operator grow their footprint? The pipeline plans a mix of brownfield phases at their existing sites plus one or two greenfield neighbours.

Stages: ExpansionWorkloadAgent parses the operator's target MW and growth horizon.

OperatorFootprintAgent loads the operator's existing site list and capacities.

DemandForecastAgent projects per-site demand under user-supplied growth assumptions.

ExpansionScoringAgent ranks expansion options by cost, lead time, and constraint headroom.

ExpansionSynthesisAgent composes the ExpansionPlan.

3. Winter Peak Stress Tester

Question: will a regional grid hold up at minus twenty-five degrees, once thirty percent of homes have heat pumps?

Stages: WinterWorkloadAgent extracts city, weather event, adoption percentage, and time horizon. WinterGridAgent loads the local feeder topology from OSM substations buffered against OEB service territory polygons, with baseload curves from IESO Demand Shape.

ColdEventSimAgent runs an hour-by-hour simulation factoring heat-pump COP loss, EV cold-weather draw, baseboard spikes, and historical cold-event catalogues. FeederRiskAgent ranks every wire and station by failure probability with stated reasons. WinterSynthesisAgent assembles a ResiliencePlan with peak number, headroom, at-risk feeder map, mitigation options, and a PASS / FAIL verdict.

Most cited at-risk feeders are older residential lines serving neighbourhoods densely packed with newly-converted heat pumps. The pipeline surfaces this consistently because the neighbourhood profile, the asset age, and the cold-weather COP curve all feed into the same scoring function.

4. Neighbourhood Electrification Readiness

Question: which neighbourhoods in our service territory are ready for full electrification, and which need help first?

Stages: ElectrificationWorkloadAgent parses the territory and adoption targets.

NeighborhoodProfileAgent loads FSA-level demographics, housing stock, and existing electric service quality. AdoptionModelingAgent projects per-FSA heat-pump and EV adoption curves under policy and incentive scenarios. ReadinessScoringAgent scores each FSA on a 0 to 100 readiness metric. ElectrificationSynthesisAgent composes the ElectrificationPlan with a ranked FSA list and targeted upgrade recommendations.

5. Climate-Adapted Grid Investment Optimiser

Question: if a utility has a fixed capital budget, how should it spend it for the most climate-resilience per dollar?

Stages: InvestmentWorkloadAgent parses the budget envelope and risk appetite.

AssetCatalogAgent loads the utility's asset inventory with condition and replacement cost.

ClimateRiskModelingAgent scores each asset's exposure to flood, wildfire, and ice-storm scenarios using NRCan and ArcGIS hazard layers. PortfolioOptimizationAgent runs a constrained knapsack solver to pick the funded set under the budget. InvestmentSynthesisAgent composes the InvestmentPlan with the funded portfolio, deferred items, and risk-buy-down metrics.

AI Inference: Claude Tool Use, Streaming, and Counterfactuals

The base agent

Every agent in the platform inherits from `backend/agents/base_agent.py`. The base class wraps Anthropic's `AsyncAnthropic` client and provides three execution modes: a structured JSON extract, a streaming text response, and a tool-using loop. Retries are bounded with exponential backoff on 429, 503, and 529 responses, and every call has a 120 second timeout.

```
class BaseAgent:
    def __init__(self, client: AsyncAnthropic, model: str | None = None):
        self.client = client
        self.model = model or settings.ANTHROPIC_MODEL

    async def ask_claude_json(self, *, system: str, messages: list[dict],
                              schema: type[BaseModel]) -> BaseModel:
        # Calls Claude with a strict JSON-output system prompt,
        # parses the response, validates against a pydantic schema.
        ...

    async def ask_claude_stream(self, *, system: str, messages: list[dict]):
        async with self.client.messages.stream(...) as stream:
            async for text in stream.text_stream:
                yield text
```

Models, by stage

- Router classification — Haiku 4.5. Sub-second response time matters and the task is short.
- Workload parsing (stage 1 of each pipeline) — Sonnet 4.6. Needs to extract structured spec fields with high precision.
- Synthesis (final stage of each pipeline) — Opus 4.7 by default. Long-context narrative generation with strict citation requirements.
- Operators can override the model per-request via the optional model field on every pipeline POST.

Tool use: the five counterfactual tools

Once a report is rendered, the planner opens a chat scoped to that report. Behind that chat is Claude with one of five tools available, depending on which pipeline produced the report. The tools are deterministic, read-only re-computations of the original analysis under different inputs.

Pipeline	Tool	Effect
Winter Peak	<code>recompute_winter_peak</code>	Re-projects peak load under new heat-pump % or EV %

Electrification	recompute_readiness	Re-scores FSA readiness under new adoption %
Expansion	recompute_expansion	Re-picks the funded set under a new target MW
Investment	reoptimize_portfolio	Re-runs the knapsack at a new budget
Siting	filter_siting_candidates	Re-filters top candidates under new LCOE, deployment, or capacity constraints

The chat loop calls the tool, gets back a fully-formatted result, and only then asks Claude to narrate. After the stream ends, a reconciliation pass scans Claude's narration for numbers that drift from the tool's actual output and rewrites them. This is the difference between "Claude says the peak is 3,500 MW" and "the simulation computed 3,710 MW".

Grounding and citations

Every synthesis prompt is built by backend/agents/grounding.py, which injects a Section 0 into the system prompt that explicitly forbids inventing quantitative values and lists the source feeds used in the current run. The synthesis agent must cite a source for every Megawatt, every percentage, every dollar figure. The frontend renders these citations as inline footnotes in the report.

Why we did not let Claude pick numbers from training data: the report needs to be defensible. A planner cannot bring "3,387 MW" to a stakeholder meeting if the model invented it. Every number traces back to PeeringDB, ECCC, IESO, AESO, OSM, StatsCan, OEB Yearbook, NRCAN, or ArcGIS Living Atlas.

Data Sources and Grid Intelligence

The services layer is where Skorpio earns its "real data, not synthetic" claim. Each file in backend/services/ owns exactly one upstream feed. Files prefixed with an underscore (_ieso_hoep_generated.py, _aeso_pool_ail_generated.py, _fsa_data_generated.py, ...) are cached snapshots regenerated by scripts in backend/scripts/, so a pipeline run does not depend on a third-party API being up at demo time.

Live feeds

Service	File	Source
IESO LMP	iso_lmp.py	Ontario Locational Marginal Pricing
ECCC weather	eccc.py	Environment and Climate Change Canada bulk_data
Carbon intensity	carbon_intensity.py	Real-time grid carbon intensity feed
Cold events	cold_events.py	Historical polar-vortex catalogue
Heat-pump COP	heat_pump_cop.py	Coefficient-of-performance curves by region and temperature
EV data	ev_data.py	EV adoption and charging data
ArcGIS hazards	arcgis_hazards.py	Esri Living Atlas: flood, wildfire, ice-storm risk
ArcGIS enrichment	arcgis_enrichment.py	Demographic and built-environment enrichment
OEB territory	oeb_territory.py	Ontario Energy Board service-territory polygons
NRCan wildfire	nrcan_wildfire.py	Natural Resources Canada wildfire risk
OpenEI	openei.py	Solar resource, facility data
NREL	nrel.py	Solar and wind resource data
Fiber latency	fiber_latency.py	Network latency to PeeringDB exchange points

Cached snapshots

- _ieso_hoep_generated.py — historical Hourly Ontario Energy Price.
- _ieso_generators_generated.py — IESO generator registry (capacity, type, location).
- _ieso_demand_shape_generated.py — historical demand-shape profiles by region.
- _aeso_pool_ail_generated.py — Alberta Pool Price + AIL.
- _fsa_data_generated.py — Forward Sortation Area demographics (StatsCan 25-10-0015 / 0021, 23-10-0308).
- _oeb_territories_generated.py — OEB utility territory polygons.

- `_oeb_utilities_generated.py` — OEB utility registry.
- `_provincial_electricity_generated.py` — provincial grid mix and capacity.
- `_provincial_generation_mix_generated.py` — generation asset mix (hydro, nuclear, wind, gas %).
- `_hdd_stations_generated.py` — Heating Degree Day meteorological stations.

Geofence on utility lookups

When a pipeline needs the utility serving a specific address, it does not name-match. It looks up the OEB territory polygon containing the point. This avoids the common failure mode where two utilities share a similar name in adjacent regions and the wrong one gets assigned.

Frontend: React, Vite, Mapbox, and the Live Reasoning UI

The frontend is a single-page React app with five report viewers, a shared composer, and a live-reasoning panel that animates the orchestrator's stage progress.

Layout

```
frontend/src/
  api/
    client.ts          # fetch wrapper, SSE EventSource helper
    types.ts           # Pipeline request / response interfaces
  components/
    PipelineLive.tsx   # Live stage progress monitor (subscribes to /api/stream)
    stageviz/          # Per-stage animations and labels
    MapView.tsx        # Mapbox interactive map; used by all 5 reports
    SitingReport.tsx   # Datacenter Siting report viewer
    ExpansionReport.tsx # Expansion report viewer
    WinterPeakReport.tsx # Winter Peak report viewer
    ElectrificationReport.tsx
    InvestmentReport.tsx
    ChatView.tsx       # SSE-streamed follow-up chat
    AuthGate.tsx       # Login wall (Google OAuth + magic code)
    DemoToggle.tsx     # Demo-mode switch (uses bundled fixtures)
  pages/
    HomePage.tsx       # Landing + pipeline picker
    PipelineRunner.tsx  # Composer + live progress + report container
  hooks/
    useJob.ts          # Poll job, subscribe to /api/stream, expose state
  demo/
    fixtures/          # 5 static JSON reports for Vercel-only demo mode
```

Live "watch the AI think"

PipelineLive.tsx opens an EventSource against `/api/stream/{job_id}` as soon as the composer submits. Each event is a small JSON payload of shape `{ stage, progress, message, paused, error }`. Progress messages stack into a chronological log underneath labelled stage cards that light up in the Skorpio orange. The whole thing was tuned so that even when the underlying pipeline takes ten minutes, the perceived state is always alive — there is always a new line of text or a stage card advancing.

Maps

- MapView.tsx is shared across all five report types. It draws candidate parcels (Siting), brownfield growth phases (Expansion), feeder risk gradients (Winter Peak), readiness heat-maps (Electrification), and funded-asset markers (Investment).
- If VITE_MAPBOX_TOKEN is not set, MapView falls back to a non-interactive coordinate table so the report is still readable.

Reports and CSV

- Each report viewer reads the pipeline's typed JSON result and renders it through Recharts and a small set of bespoke layout components.
- The reports table on the home page supports sort, filter, CSV export, and CSV import with conflict resolution. Imported rows are tagged `imported_from_email` so collaborators can see what came from where.
- A guided tour runs ten steps on first load: dot pagination, Skip and Next pill buttons, anchor-click-to-advance, ResizeObserver to keep highlights aligned, and a (?) replay button to re-trigger the tour later.

Database, Auth, and Storage

Models

- User — Skorpio account. Created lazily on first successful Google OAuth or magic-code login. Holds email, display name, and google_sub for OAuth.
- Job — One row per pipeline run. Stores spec (request payload), pipeline_id, stage, progress, message, error, result (final report JSON), progress_log (replayable on re-open), and timestamps.
- EmailCode — Magic-code login record. The 6-digit code is hashed with HMAC-SHA256, has a 10-minute TTL, and is single-use.

Auth

- Google OAuth 2.0 at /api/auth/google/login. Standard authorisation-code flow with PKCE.
- Magic-code email at /api/auth/email/request-code -> /api/auth/email/verify-code. Codes are sent through Resend; the API returns a JWT session on verify.
- Sessions are stateless JWTs (HS256, signed with SKORPIO_AUTH_SECRET). Default TTL is 24 hours. No revocation list — short TTLs and a forced re-login are good enough for the demo perimeter.
- An optional email allow-list (SKORPIO_ALLOWED_EMAILS) gates demo access; empty means open.

Migrations

Alembic 1.13 manages the schema. alembic.ini at the project root points at backend/migrations/. Migrations run on container start through a small bootstrap in main.py's lifespan.

Observability and Reliability

- Request logging — every API request is logged with method, path, status, latency, and a short request_id. Errors carry the full traceback.
- Progress log replay — every progress update is appended to the job's progress_log column, so re-opening a completed job in the UI replays the live stages exactly as they happened.
- Retry policy — base_agent.py retries 429, 503, and 529 with exponential backoff (5s, 15s, 30s) before failing the stage.
- Cancellation safety — JobControl checkpoints at every stage boundary. Mid-stage cancellation raises JobCancelledError and writes a clean cancelled status to the job row, so the frontend never sees a half-finished report.
- Graceful Snowflake handling — the optional analytics path skips itself with a single log line if Snowflake credentials are absent, so the API boots clean on a developer laptop.

Engineering Trade-offs in 72 Hours

- Five pipelines in one orchestrator — we chose breadth over depth. Each pipeline could be a startup. We shipped all five at hackathon-grade quality rather than one at production-grade.
- Asyncio in-process over a worker pool — Celery / RQ would have added containers, a broker, and serialisation boundaries. For five concurrent jobs at a hackathon demo, asyncio was simpler and never blocked us.
- Cached data snapshots — every "`_*_generated.py`" file is a snapshot regenerated from the source feed. We pay an hour of regeneration once a day to guarantee the demo never depends on an external API being up.
- Custom Langflow components — wrapping the siting pipeline in Langflow components gave us a path to a visual builder without abandoning code-first agents. We did not go further than siting because the other four pipelines are evolving too fast.
- Demo fixtures bundled with the frontend — the Vercel build can present a complete experience without a backend, which is what gets the link past corporate VPNs at demo time.
- Snowflake left in the tree but inert — the original platform vision included Snowflake-backed analytics. Removing the dead code is on the post-hackathon list, but it does not affect demo or prod paths.

Roadmap and Post-Hackathon Plans

Immediate (week 1)

- Remove the dead Snowflake stack (`snowflake_client.py`, `snowflake.py` router, `snowflake_analytics.py`, lifespan hook, `.env` keys). Boots faster, smaller Docker image, no "Snowflake init skipped" warning.
- Bring HACKATHON_TODO.md to its archived home at `docs/history/`.
- Wire automated smoke tests into CI for all five pipelines against the test DB.

Short term (month 1)

- Dashboard tab. Currently a placeholder. Either ship cross-run aggregates (average runtime per pipeline, top-rated sites this week, budget burn-down) or remove from the sidebar.
- Multi-tenant: scope jobs by `owner_email` and enforce in the API layer. The schema already supports it.
- Live IESO feed refresh: replace the cached snapshot with a 5-minute polling job that updates the carbon-intensity and LMP tables in-place.

Medium term (quarter 1)

- Pipeline-to-pipeline composition: a siting report whose result feeds an investment optimiser run for the chosen region.

- Custom Mapbox styles to lift the Skorpio dark theme into the map tiles themselves.
- Replace the polling SSE with WebSockets for sub-100ms progress updates.
- Open-source the grid/ module as a standalone Python package.

Team, Credits, and Closing Notes

Name	Role	What they built
Christian Rafael	Pitch and Strategy	Led the pitch, the video script, and the printed dossiers. Owned the narrative arc that ties the five pipelines together.
Fawad Arshad	Backend	Built the orchestrator and the first wave of agentic pipelines. Owned the stage pattern that all five pipelines now share.
Krish Barot	Backend	Built the remaining pipelines and the services layer. Owned the live grid feeds and the cached-snapshot regeneration scripts.
Maximilian Ali	Frontend	Shipped the chat, the live dashboard, and the five report viewers. Owned the SSE-streamed live-reasoning UI.
Vansh Barot	UI / UX Design	Designed the visual system, the auth flow, and every detail of how the reports look.

Credits

- Anthropic for Claude (Opus 4.7, Sonnet 4.6, Haiku 4.5) and the AsyncAnthropic SDK.
- IESO, AESO, ECCC, OEB, StatsCan, OSM, NRCan, and ArcGIS Living Atlas for the data feeds that make every report defensible.
- Mapbox for the map tiles and styling.
- Vercel for the demo hosting.
- The Seneca Hackathon 2026 organisers and judges.

Closing

Three sleepless days. One platform. Five tools that share the same brain. Skorpio.